Documentación Técnica: Simulador de Combate Espacial

Especificaciones de desarrollo y arquitectura POO para consola en C++

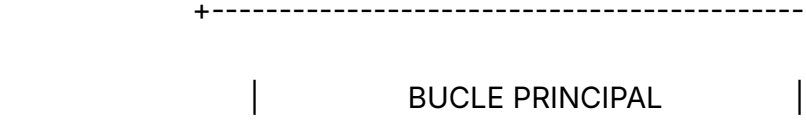
1. Información General y Requisitos del Sistema

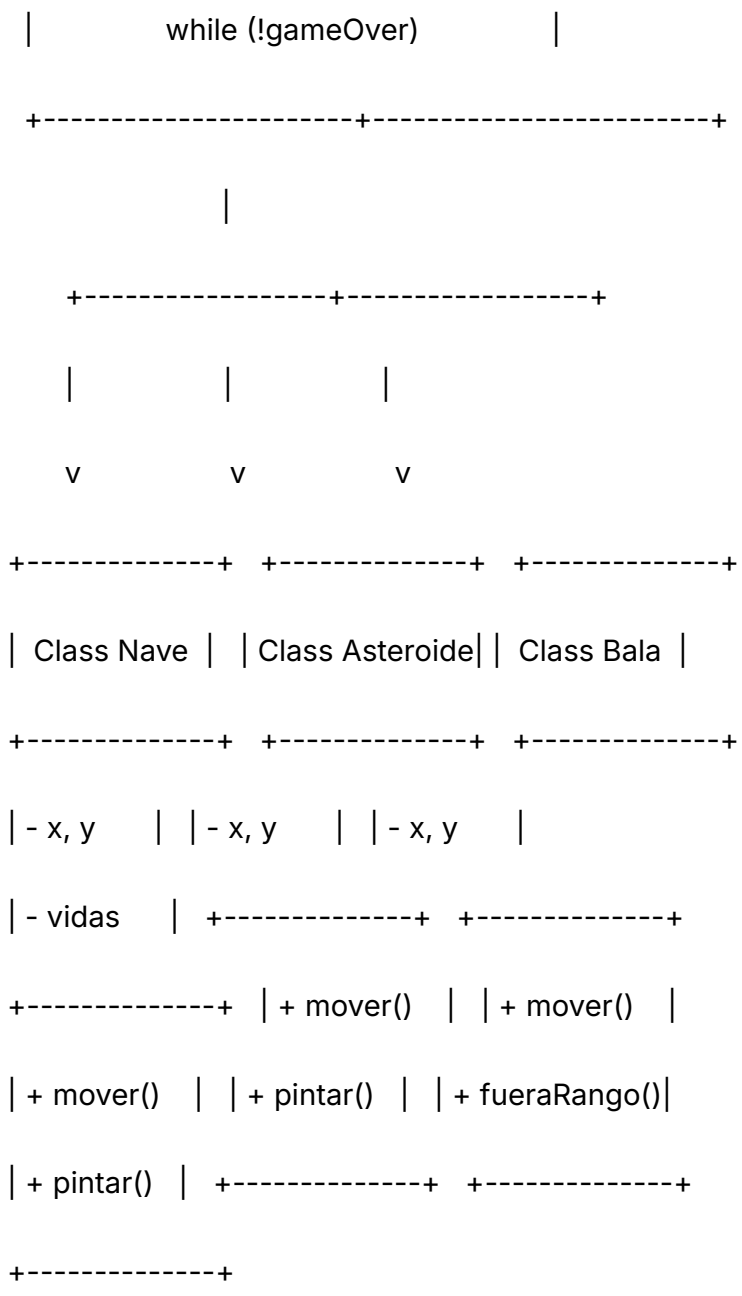
Parámetro	Especificación
Proyecto	Simulador de Combate Espacial en Consola
Materia	Laboratorio de Programación (LPR) — 5° Año (E.E.S.T. N° 1)
Lenguaje	C++ (Estándar C++11 o superior)
Entorno Recomendado	MinGW / GCC, Visual Studio Code, Dev-C++
Librerías del Sistema	`<iostream>`, `<windows.h>`, `<conio.h>`, `<vector>`, `<ctime>`, `<cctype>`

---

2. Arquitectura y Estructura del Código Fuente (`src/main.cpp`)

El sistema está diseñado bajo el paradigma de **Programación Orientada a Objetos (POO)**. Se divide en tres entidades principales encargadas de la representación, actualización de estado y renderizado mediante caracteres en consola.





2.1. Clase `Nave` (El Jugador)

Encapsula la entidad controlada por el usuario, administrando su posición espacial y sus puntos de vida.

- **Atributos Privados (`private`):**
  - `int x, y`: Coordenadas cartesianas actuales en la consola de comandos.
  - `int vidas`: Contador entero que almacena la salud restante del jugador.

- **Métodos Públicos (`public`):**
    - `Nave(int _x, int _y, int _vidas)`: Constructor que inicializa la ubicación de origen y la salud inicial.
    - `int getX()`, `int getY()`, `int getVidas()`: Métodos de acceso (*getters*) para consulta externa preservando el encapsulamiento.
    - `void reducirVida()`: Decrementa de forma controlada el contador de salud ante colisiones.
    - `void pintar()` y `void borrar()`: Realizan el dibujado y borrado del carácter de la nave en pantalla utilizando la función `gotoxy()`.
    - `void mover(char tecla)`: Procesa la tecla ingresada, valida los límites de pantalla de la consola y actualiza las coordenadas correspondientes.

2.2. Clase `Asteroide`

Representa los obstáculos móviles que descienden verticalmente hacia la nave.

- **Atributos Privados (`private`):**
    - `int x, y`: Coordenadas de posición actual en pantalla.
- **Métodos Públicos (`public`):**
    - `Asteroide(int _x, int _y)`: Constructor para definir el punto de aparición inicial.
    - `int getX()`, `int getY()`: Métodos *getter* de posicionamiento.
    - `void pintar()`, `void borrar()`: Dibuja y borra la representación gráfica (carácter `'0'`) en las coordenadas actuales.
    - `void mover()`: Incrementa la coordenada y. Si el asteroide sobrepasa el límite inferior de la pantalla, se reinicia en la parte superior con una coordenada x generada aleatoriamente.

2.3. Clase `Bala`

Gestiona los proyectiles emitidos por el jugador para neutralizar obstáculos.

- **Atributos Privados (`private`):**
    - `int x, y`: Coordenadas cartesianas del proyectil.
- **Métodos Públicos (`public`):**
    - `Bala(int _x, int _y)`: Constructor que posiciona el proyectil en el punto de disparo de la nave.
    - `int getX()`, `int getY()`: Métodos de acceso a la posición.

- `void mover()`: Desplaza el proyectil hacia la parte superior reduciendo la coordenada y.
- `bool fueraDeRango()`: Evalúa si el proyectil ha superado el margen superior para autorizar su liberación de memoria.

---

3. Funciones Auxiliares del Motor

Para coordinar la salida gráfica en la terminal de Windows sin el parpadeo derivado de limpiezas de pantalla completas (`cls`), se emplean las siguientes funciones auxiliares:

- `void gotoxy(int x, int y)`
  Aplica la API de Windows mediante `SetConsoleCursorPosition` sobre el manejador de la consola (`STD_OUTPUT_HANDLE`) para reubicar el cursor de texto en coordenadas precisas.
- `void ocultarCursor()`
  Ajusta la propiedad `bVisible` de la estructura `CONSOLE_CURSOR_INFO` a `FALSE`, evitando la visibilidad del cursor parpadeante durante el ciclo de ejecución.

---

4. Gestión de Memoria y Ciclo Principal

El programa aplica **gestión explícita de memoria dinámica** mediante contendores estándar:std::vector<Asteroide*> asteroides;

std::vector<Bala*> balas;

Principios de Implementación

1. **Asignación y Liberación:** Los objetos se instancian en el *heap* utilizando el operador `new`. Al ser destruidos o salir del límite de pantalla (`fueraDeRango()`), se debe llamar al operador `delete` y remover el puntero del vector para prevenir fugas de memoria (*memory leaks*).
2. **Entrada No Bloqueante:** El bucle principal (`while (!gameOver)`) ejecuta la lectura de teclado mediante `_kbhit()` y `_getch()`, permitiendo actualizar el

estado de los objetos y renderizar fotogramas de forma continua sin pausar la ejecución.

3. **Detección de Colisiones:** En cada iteración se verifican las intersecciones geométricas entre la posición de los proyectiles y los asteroides, así como la colisión entre los asteroides y la nave, decrementando las vidas hasta alcanzar la condición final (`vidas == 0`).